

JobIQ — Project Technical & Interview Master Guide

End-to-End System Architecture, Codebase Walkthrough & Interview Q&A;

1. Executive Overview & Problem Statement

JobIQ is an AI-powered job aggregation and career operating platform specifically designed to aggregate, deduplicate, score, and track software engineering, internship, and hackathon opportunities in India (INR context).

Key Problems Solved:

- **Fragmented Opportunity Landscape:** Students & developers check multiple websites (LinkedIn, Internshala, Unstop, Naukri) daily.
- **Irrelevant / Low Quality Data:** Traditional aggregators contain generic templates, duplicate listings, and inaccurate salary ranges.
- **Lack of Smart Matching:** Candidates struggle to know which jobs match their exact tech stack.
- **Scattered Application Management:** No single dashboard to track pasted custom job links alongside scraped opportunities.

2. Full Technology Stack

Layer	Technologies Used	Key Responsibilities
Frontend UI	React 18, TypeScript, Vite, Tailwind CSS, Lucide Icons	Single Page Application (SPA), responsive dashboard, Kanban pipeline, dynamic filtering
State & Data	Zustand (Global Auth/User), React Query (TanStack Query v5)	Client-side cache management, optimistic updates, JWT token persistence
Backend API	Python 3.9, FastAPI, Pydantic v2, Uvicorn ASGI	Asynchronous REST API endpoints, input validation, dependency injection authentication
Database & ORM	SQLite (jobiq.db), SQLAlchemy 2.0 ORM	Relational data storage, indexing, declarative models, relationship cascades
Data Pipelines	BeautifulSoup4, HTTPX, Custom Scrapers, difflib	Live web scraping, rate-limiting, range numeric extraction, fuzzy deduplication
Task Queue	Celery, Redis Task Broker	Asynchronous background worker execution for scheduled scraping & heavy matching tasks
ML / Data Science	Scikit-Learn (TF-IDF Vectorizer), Cosine Similarity, Regex Taxonomy	Fast content-based job matching, 200+ skill taxonomy extraction, resume parsing

3. High-Level System Architecture & Flow

The application follows a clean 3-tier decoupled architecture:

- 1. **Client Layer (Browser):** React SPA renders the AppShell, Opportunities Grid, Kanban Pipeline, and Resume Upload view. Communicates via Axios client (`/api/v1`) with JWT Bearer header.
- 2. **Application Layer (FastAPI):** Handles authentication, routes request validation with Pydantic, triggers sync link parsing, and dispatches background scrape jobs to Celery.
- 3. **Data & ML Engine:** Periodically pulls listings from external web sources, computes TF-IDF similarity vectors between resumes and opportunities, and updates match scores in `jobiq.db`.

4. Database Schema & Data Models

Table Name	Key Columns	Description & Purpose
users	id, email, hashed_password, full_name, target_role, location_preference	User account details, authentication credentials, and user preferences
opportunities	id, title, company, location, salary_min, salary_max, stipend, prize_pool, description, required_skills, type, source, source_url	Central opportunity store. Indexed by title, company, type, and source_url
job_matches	id, user_id, opportunity_id, match_score (0-100), matching_skills, missing_skills, match_reason	Pre-computed ML job match results linking users to opportunities
applications	id, user_id, opportunity_id, status (interested/applied/interviewing/offered/rejected), applied_at, notes	Kanban pipeline job application tracker for users
custom_jobs	id, user_id, title, company, source_url, location, status, salary_min, salary_max	User-pasted custom jobs parsed via the Smart Link Parser service
resumes	id, user_id, raw_text, extracted_skills, experience_summary, education_summary	Parsed user resume data used as baseline vector input for the ML matcher

5. Deep Dive into Core Technical Modules

A. Web Scraping & Ingestion Engine (`app/scrapers/`)

JobIQ uses lightweight HTTP requests (via `httpx` and `BeautifulSoup4`) rather than heavy browser automation like Selenium to ensure high speed, low memory usage, and reliability:

- **LinkedIn Scraper (`linkedin\_scraper.py`)**: Leverages the LinkedIn Guest Search API (<https://www.linkedin.com/jobs-guest/jobs/api/seeMoreJobPostings/search>) to fetch real, direct job links without login requirements.
- **Unstop Scraper (`unstop\_scraper.py`)**: Queries Unstop's public API for competitions & hackathons. Categorizes listings strictly as *hackathon* or *competition*, excluding them from standard job pipelines.
- **Internshala Scraper (`internshala\_scraper.py`)**: HTML parser targeting tech internships in India, extracting title, company, stipend, duration, and direct application links.
- **Deduplication Engine (`deduplicator.py`)**: Employs `difflib.SequenceMatcher` fuzzy matching combining title similarity (60% weight) and company similarity (40% weight). Soft-deletes listings if similarity exceeds 85%.

B. Smart Link Parser Service (`app/services/link\_parser.py`)

When a user pastes any job link into JobIQ, the **LinkParser** executes:

1. Fetches HTML content with custom User-Agent headers.
2. Extracts OpenGraph meta tags (`og:title`, `og:description`, `og:site\_name`) and schema.org JSON-LD data.
3. Extracts location, required skills, and salary numbers.
4. Returns a clean JSON preview for user verification before saving to the Kanban pipeline.

C. AI / ML Matching Engine (`app/ml/`)

Rather than relying on slow LLM calls for thousands of jobs, JobIQ uses a hybrid **Multi-Factor Scoring Engine**:

Final Match Score Formula:

$$\text{Score} = (\text{Skills_Overlap} \times 0.40) + (\text{TFIDF_Text_Similarity} \times 0.30) + (\text{Salary_Match} \times 0.15) + (\text{Location_Match} \times 0.15)$$

- **Skills Overlap (40%)**: Exact regex matching against a 200+ technology taxonomy (`skill\_extractor.py`).
- **TF-IDF + Cosine Similarity (30%)**: Uses Scikit-Learn's `TfidfVectorizer` to convert resume text and job description into sparse TF-IDF vectors and compute cosine similarity.
- **Salary & Location Alignment (15% each)**: Evaluates user preferences against job constraints.

6. Top Technical Interview Questions & Answers

Q1: Walk me through the architecture of your JobIQ project.

JobIQ is built with a decoupled FastAPI + React TypeScript architecture. The frontend uses React Query and Zustand for asynchronous state management. The backend handles job aggregation via scrapers (LinkedIn Guest API, Unstop, Internshala), deduplicates listings using fuzzy string matching, and scores jobs against user resumes using a Scikit-Learn TF-IDF and taxonomy-based multi-factor matcher.

Q2: How do you handle web scraper reliability and prevent IP bans?

We use direct HTTP endpoint requests with custom User-Agent headers instead of heavy browser automation (Selenium). We implement rate-limiting sleeps in our BaseScraper class, process requests asynchronously, and fallback to direct endpoint calls when browser automation fails.

Q3: How does your fuzzy deduplication algorithm work?

We use Python's `difflib.SequenceMatcher` to compute a weighted string similarity metric between listings: 60% weight on title similarity and 40% on company name similarity. If the combined score meets or exceeds 0.85 (85%), the listing is flagged as a duplicate and soft-deleted in SQLite.

Q4: Why did you choose TF-IDF + Cosine Similarity over LLMs/Transformers for job matching?

Speed, scalability, and cost. Running heavy Transformer embeddings (e.g. BERT) or LLM calls over thousands of job descriptions per user query takes seconds/minutes and is costly. TF-IDF vectorization with Scikit-Learn executes in milliseconds, uses minimal memory, and delivers precise content matching for tech skill keywords.

Q5: How did you handle bad/corrupted data like stipend range strings ('11,000 - 21,000')?

We wrote a robust regex numerical extraction helper (`_safe_int`) in `base.py` that parses the first valid numeric group (`\b\d+(?:,\d+)*\b`) from range strings rather than stripping non-digits blindly, which previously caused concatenation bugs. We also executed database cleanup scripts to normalize existing data.

Q6: How does the Smart Link Parser work when a user inputs an arbitrary URL?

The LinkParser uses HTTPX to download the target HTML page and parses standard metadata tags: OpenGraph (`og:title`, `og:description`), Twitter cards, and schema.org JSON-LD microdata. It extracts title, company name, location, and description, presenting a preview modal to the user before inserting it into their application pipeline.

Q7: How did you separate hackathons/competitions from standard software jobs?

In `unstop_scraper.py`, we explicitly filter items by type. Anything tagged as competition or hackathon is stored with `OpportunityType.HACKATHON` or `COMPETITION` in the database, allowing frontend tabs to strictly separate job listings from competitions.

7. Project Folder Structure & Quick Start

```

jobiq/ ■■■■ jobiq-backend/ ■ ■■■■ app/ ■ ■ ■■■■ core/ # Config, Database engine, JWT
Auth, Bootstrap ■ ■ ■■■■ ml/ # Skill Extractor, Resume Parser, TF-IDF Matcher ■ ■ ■■■■
models/ # SQLAlchemy DB Models (Opportunity, User, Application...) ■ ■ ■■■■ routes/ #
FastAPI Routers (/opportunities, /auth, /applications...) ■ ■ ■■■■ schemas/ # Pydantic
Schemas for Request/Response validation ■ ■ ■■■■ scrapers/ # Scraper base class,
LinkedIn, Unstop, Internshala, Deduplicator ■ ■ ■■■■ services/ # Opportunity Ingestion,
Link Parser service ■ ■ ■■■■ tasks/ # Celery Background Tasks & Cron Schedules ■ ■■■■
requirements.txt # Python Dependencies ■ ■■■■ jobiq.db # SQLite Database ■■■■
jobiq-frontend/ ■■■■ src/ ■ ■■■■ api/ # Axios API Clients & Endpoints ■ ■■■■ components/ #
UI Components & PasteJobModal ■ ■■■■ pages/ # Dashboard, Opportunities, Pipeline,
Resumes, InterviewPrep ■ ■■■■ store/ # Zustand Auth & State Stores ■ ■■■■ AppShell.tsx #
Main Application Navigation Shell ■■■■ package.json ■■■■ vite.config.ts

```

Quick Commands to Demonstrate/Run in Interview:

- **Start Backend:** cd jobiq-backend; uvicorn app.main:app --reload (Runs on http://127.0.0.1:8000, API Docs at http://127.0.0.1:8000/docs)
- **Start Frontend:** cd jobiq-frontend; npm run dev (Runs on http://localhost:5173)
- **Trigger Live Scrape via API:** Send POST /api/v1/admin/scrape

End of JobIQ Master Interview Guide